LLVM BOLT 中的自适应 CMOV 优化

1 问题描述

1.1 背景与问题

在 X86 的处理器中，我们常常使用 CMOV 命令来替代现有的 if-else 语句，以消除分支预测失败带来的流水线冲刷惩罚。简单而言，当我们的 CPU 流水线遇到一个 if-else 或 jcc 指令时，就会产生一个问题，当条件结果还没算出来的时候，我们不知道执行哪一条分支的指令，这时，为了不让流水线停下来空等，CPU 会引入一个“分支预测器”，提前预测一个方向，然后直接顺着猜的路去取指、译码，甚至提前执行。

但是，当我们的预测失败时，就会导致指令失效，进而引发不必要的消耗，CMOV 指令的巧妙之处在于它消除了分叉，也就是 CMOV 是无论条件满不满足，两条路的结果我都算出来放在寄存器里，最后根据条件，用一条指令直接把正确的结果放进目标寄存器。

然而，CMOV 并非没有代价：

1.无条件的数据依赖：CMOVcc 必须等待条件标志位（EFLAGS）和两个源操作数（Source Operands）全部就绪才能执行，即使该条件在运行时并不成立。

2.串行化关键路径：当 CMOV 的更新结果作为下一次循环迭代的输入时（循环携带依赖），它会把可能耗时极长的慢速操作（如 Cache Miss 的 Load 指令）强行串接到循环的关键路径上。

当条件分布极度偏置（如 95% 的情况条件为真或假）时，分支预测器可以极高地准确预测走向，此时分支的预测开销趋近于零，而 CMOV 引入的串行依赖链代价却依然全额支付。此时，将高度偏置的 CMOV 恢复为分支，可以获得显著性能收益。

1.2 现有工具局限性

尽管 LLVM 编译后端（X86CmovConverterPass）具备反向转换的能力，但受限于编译期的静态视角，会产生以下问题：

1.极度保守的静态阈值：为了避免性能退化，LLVM 后端往往设置极其严苛的成本模型（例如要求预期收益必须覆盖 25% 的误预测惩罚），导致大量真正高度偏置的场景无法被优化。

2.正如我们上文所说，BOLT 现有 Pass 是将条件分支转换为 CMOV，BOLT 作为基于运行时画像（Profile-Guided）的后链接优化器，主线自带的 CMOVConversion 通道方向是“分支转 CMOV”，且其决策依据是 LBR 采集的误预测率，专门用于拯救预测不准的分支。而对于“预测极准但依赖链累赘”的 CMOV，BOLT 目前缺乏主动干预的机制。

2 优化设计

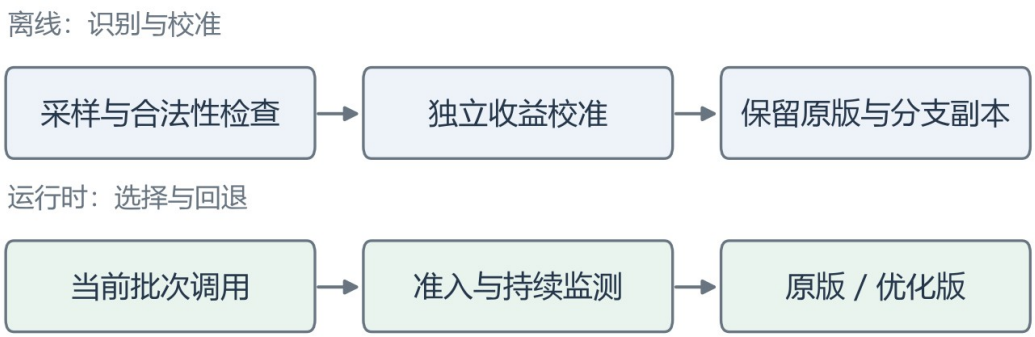


图 1 优化总体框架

对上述微架构痛点与工程挑战，本项目在主线 LLVM BOLT 中设计并实现了一个优化功能，也就是在 BOLT 中，对已有 CMOV 提供基于采样与独立校准的反向转换，并结合双版本执行、运行时准入和持续回退机制，使生成的程序能够适应当前输入，在获取收益的同时控制不适用输入下的额外开销。优化设计可以简单概括为以下几点：

1.基于采样与独立校准的反向转换：利用 BOLT 插桩采集真实的谓词方向计数（而非仅靠误预测率），识别高度偏置的 CMOV 组。在生成转换前，通过独立的数据集进行严苛的配对计时校准，确保仅在收益确凿时才执行“CMOV→分支”的转换。

2.双版本执行与运行时准入：在同一 ELF 中同时保留 CMOV 原始版与分支优化版。在批次处理的安全边界引入运行时守卫（比较真实批次的执行耗时），通过保守的准入协议，决定当前调用是否真正进入优化版。

3.持续回退与输入适应：一旦运行时准入失败，或在连续执行中发现分布漂移导致收益消失，控制器将永久放弃该次调用的优化，安全回退至 CMOV 路径。以此将不可控的分布漂移风险，转化为可监测、可回退的执行开销。

2.1 分层准入策略

为了将上述思路落实到实际程序中，我们将准入分为四层：先检查改写是否合法，再观察条件是否偏置，随后通过独立校准确认收益，最后由当前调用决定是否进入优化路径。前两层用于寻找机会，后两层用于验证机会，避免将“方向偏置”直接等同于“执行更快”。

表 1 分层准入条件

阶段	主要条件
静态检查	循环内 2～4 条同条件、纯寄存器 CMOV；局部编码不增长，满足内核复制和批次入口要求。
方向采样	条件样本不少于 1 万次，主导方向占比不低于 95%。
独立校准	使用独立输入，要求候选加速比的 95%区间下界不低于 1.02。
运行时准入	普通任务至少 512 批，每批 4096 条查询；连续 3 对试测均显示优化版耗时低于原版的 90%。

这里的 95%是指目标条件的真假次数比例，并非 95%的查询具有某种输入特征，也不是处理器实际的预测准确率。偏置只用于离线筛选；当前运行时控制器比较真实批次的耗时，不直接读取分支预测器状态。

简单而言，这四层判断依次关注功能等价、转换机会、独立收益和当前调用的适用性。这样可以将静态合法性与动态收益分开，避免只凭一个偏置阈值就修改程序。

独立校准使用加速比区间的下界，例如中位加速比虽然大于 1，但区间下界不足 1.02，仍然不批准正式转换。这 2%的余量用于筛除收益较小或波动较大的候选；它是工程准入规则，不是对后续任意输入的性能保证。

我们在运行时还设置了批次规模门槛，是因为计时和分发本身需要成本。若任务很短，即使分支副本的内核略快，也可能无法抵消这些成本。因此，不满足规模条件的任务直接使用原路径，不启动 A/B 试测。当前要求每批恰好 4096 条查询，便于按批比较成本，也构成了本实现的适用范围。

2.2 运行时监测与回退

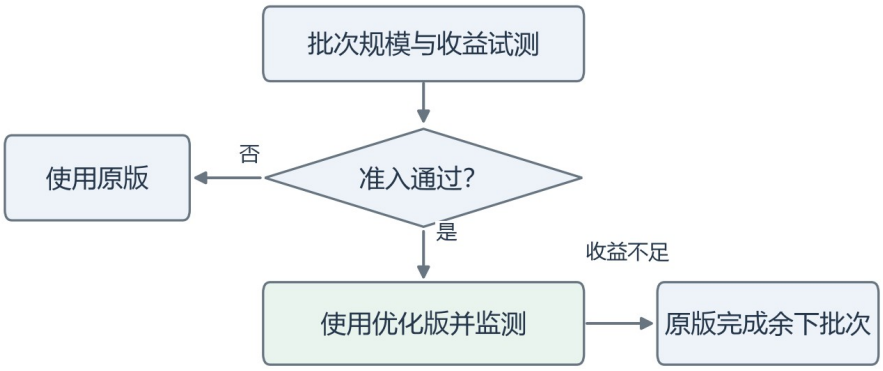


图 2 运行时执行选择

我们将原始 CMOV 实现记为 A，分支副本记为 B。试测时，A 和 B 依次处理相邻的真实批次，完成的结果直接累加，不重复计算同一批。例如 A 耗时 100 μs、B 耗时 85 μs，这一对通过；若 B 耗时 95 μs，则放弃准入，由 A 完成余下工作。

初始准入最多检查 3 对，任何一对没有达到 10%的耗时优势就立即退出，只有三对全部通过才持续执行 B。A/B 处理的是相邻而非完全相同的数据，因此这一判断依赖附近批次工作量具有可比性。保守门槛可以减少误准入，但也可能放弃一部分有收益的调用；我们选择优先控制额外开销。

进入 B 后，每两批共用一次起止计时，每执行 16 批 B 再安排一批真实工作给 A。若最近三组 B 的中位耗时达到参照 A 的 95%，控制器就重新测量 A；必要时用三批 A 的中位数确认。收益不足时，本次调用不再试探 B，下一次调用重新判断。

这里的 90%和 95%承担不同作用：90%是进入优化版的严格门槛，95%是持续执行时的收益警戒线。例如参照 A 仍为 100 µs，B 从 85 µs 上升到 96 µs 时，即使尚未慢于 A，也会触发确认，提前检查收益是否已经不足。重新测量当前 A，则避免一直用调用开始时的旧参照判断后半段输入。

确认过程也不把已完成的工作重跑一遍，而是将接下来的真实批次交给 A。先测一批；若仍显得不利，再补测两批取中位数，减少偶然偏快的 A 样本造成误回退。确认收益不足后，本次调用剩余部分固定使用 A，不再反复切换。

这一设计用较少的读钟操作换取持续观察能力，但仍存在试测成本和发现变化的延迟。因此，性能保护的目标是控制不适用输入的额外开销，具体效果由完整调用计时验证。

具体而言，两批合并计时减少了读钟次数，但最近三组成本可能覆盖六批 B，输入突变后需要一定工作量才能发现。分布切换实验因此不是附加演示，而是用来检查“前半段有收益、后半段不再适合”时，前期收益能否覆盖检测和回退成本。

3 实现细节

3.1 组级改写与双版本生成

在 BOLT 中，CMOVBranchify 负责识别满足条件的指令组，保留原始内核，并在副本中完成“CMOV→条件跳转与 MOV”的转换。对于共享同一条件的多条 CMOV，一条跳转统一控制整组赋值，避免为每条指令分别生成分支。

```
for (MCInst &Inst : *Moves) {
  unsigned CC;
  const bool Converted =
      MIB.convertConditionalMoveToMove(Inst, CC);
  assert(Converted && CC == S.CC);
}
```

以上为核心片段，省略了外围的复制、跳转生成和合法性检查。转换保持组内赋值顺序，以保留寄存器之间原有的依赖；先检查新旧编码大小，再接入分支副本。这里的 CMOV 只完成候选值选择，候选值由前序代码准备，并不意味着 CMOV 指令本身执行完整的两段 if-else 代码。

对这段代码可以分三步理解：

1.遍历指令组：逐条取得副本中的条件移动，只在已通过合法性检查的组内执行转换，原始内核保持不变。

2.转换赋值：将条件移动改为普通 MOV，并取出原条件码 CC。赋值顺序沿用原指令组，保留组内寄存器依赖。

3.统一控制：通过断言复核各指令使用相同条件，外围跳转决定是否执行整组 MOV，组内不再重复生成条件分支。

例如最终测试内核的两条 CMOV 分别更新区间起点和长度，编码共 8 字节。分支副本使用 2 字节条件跳转和两条各 3 字节的 MOV，局部仍为 8 字节。这里约束的是被替换的指令组，而不是整个二进制：保留原内核、分支副本和运行库仍会增加文件内容。

在功能正确性上，我们只接受纯寄存器赋值，并保持原赋值顺序，不引入额外访存或修改栈指针。若第二条赋值使用第一条更新后的寄存器，顺序就直接影响结果，不能为了布局而交换。不能满足复制、控制流或异常处理条件的内核保留原代码。

在接入方式上，当前实现要求程序提供明确的内核、原始批处理函数和转发入口。BOLT 检查入口确实以单条尾跳转转发到原批处理函数，再将它接入选择逻辑；调用方仍使用原入口。运行库只在相应选项开启且检查通过后链接，不依靠全局初始化钩子启动。

此外，画像记录与输入二进制的 SHA256、函数位置和 CPU 信息绑定。这样可以防止把另一份程序或机器上的采样结果误用于当前改写。生成后的程序换机器运行时，仍需要重新验证性能，画像绑定不能代替跨平台实验。

3.2 批次执行与异常回退

cmov_observe.cpp 负责执行真实批次并记录耗时。Measured 在完成工作后推进内部游标，Fallback 则将已累加的结果和未完成位置交回原始路径。下面的判断同时处理计时异常和收益不足：

```
if (!Measured(Original, Reference)) return Fallback(1);
if (!Measured(Branch, Trial)) return Fallback(1);
if (above_ratio(Trial, Reference, 90))
  return Fallback(2);
```

因此，发生回退时不会重新执行已经完成的批次，也不会遗漏剩余任务。输入上下文在调用期间只读，各次调用的计数独立保存，避免通过共享状态影响功能结果。

为了说明“恰好处理一次”是如何保证的，可以将内部状态理解为两个量：Sum 保存已经完成的结果，Next 指向第一个尚未处理的批次。执行一批后先累加结果、推进游标，再检查结束计时。因此，即使结束读钟失败，该批工作也已经记入 Sum，回退从更新后的 Next 继续。若开始读钟失败，游标没有前进，原路径会正常处理这一批。

耗时判断使用线程 CPU 时间，尽量减少线程被调度出去的等待对比较的影响；但系统调用读钟仍有开销，WSL 环境也仍有波动。两批合并计时后，记录的是整组耗时除以批数，再对最近三组取中位数，而不是对每条查询单独计时。

3.3 零工作路径优化

对于空索引或空查询，我们进一步检查机器指令是否能够证明返回值为 0，且该路径没有写内存、调用等副作用。本接口将各批次的查找位置累加；只有满足上述证明，才允许在入口一次返回 0，省去逐批调用和运行时观察。

未知前缀、不同返回常量或不满足接口条件的程序保留原代码。零工作判断与普通任务准入分开处理，使本来无需计算的任务不承担观察成本。

例如空索引下，512 批查询各自的查找位置都为 0，原接口的结果就是 512 个 0 相加。若已证明这些调用没有其他副作用，就可以一次返回 0。相反，若内核在空输入时还会更新统计信息或返回其他常量，即使看起来“没有查找”，也不能直接省略调用。

检查顺序同样影响安全性：先识别空批次和空查询，再访问确实需要的上下文；只跳过已经证明为 0 的前缀。这样既避免对未使用的上下文作多余访问，也将普通任务的性能判断与零工作语义分开。

4 实验验证与结果分析

4.1 实验设置

实验在 Ryzen 7 7735H、WSL2、Linux x86-64 环境中进行，进程绑定来宾逻辑 CPU 2，以减少迁移带来的波动。该设置不代表独占宿主物理核心。负载由 Clang 18 按正常-O3 编译，BOLT 固定为 2026 年 11 月 2 日主线提交。

对照版与优化版使用相同输入和通用 BOLT 参数，包括原有的-cmov-conversion；差别在于是否开启-cmov-branchify-runtime。训练、独立校准和正式测试分开，避免用测试输入决定转换。

正式测试包含 52 项常规负载和 7 项边界，共 59 项、31 轮、14632 次计时执行。各版本按配对方式交错运行，每次校验输出。以下加速比为“关闭耗时÷开启耗时”，以配对比值的中位数和逐项 95%区间报告；误差线不代表所有负载的联合置信范围。

这里的交错运行，是让相同输入在相邻时间段分别运行各版本，并按配对块计算比值，而不是先测完全部关闭版、再测全部开启版。每个块包含 8 次运行，其中保留辅助参照和关闭版重复对照，以便复查顺序与环境波动。正文聚焦最终开关对照，原始运行记录完整保存。

4.2 强偏置负载性能

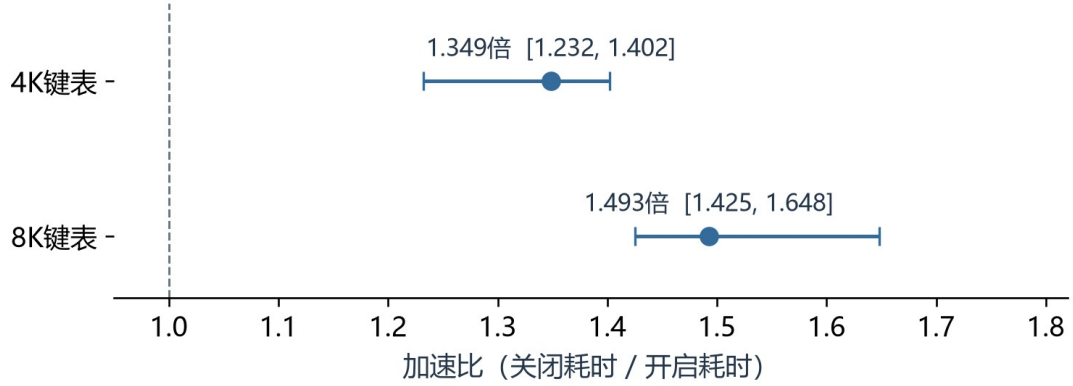


图 3 强偏置负载的加速比与 95%区间

4K 和 8K 键表分别获得 1.349 倍、1.493 倍加速，耗时约减少 25.86%和 33.01%。换算为相同工作量，原来 100 ms 的任务分别约需 74.1 ms 和 67.0 ms。两个区间均高于 1，表明收益超过了本次重复计时的波动范围。

这一结果对应于前文的优化动机：方向稳定的输入提供了分支执行机会，保守准入使 B 版本承担更多工作。8K 在本组数据中收益更高，但不能据此推导“键表越大越快”；查找深度、缓存和执行依赖的影响需要分别验证。

从实际路径看，31 轮中每种规模各有 62 次优化版测量，4K/8K 分别有 57 次和 58 次通过运行时准入。也就是说，较高的条件偏置并没有强制每次都进入 B；个别调用仍会因当时的耗时比较而留在 A。最终加速比已将这些调用以及试测成本包含在内。

再看分布切换负载：4K/8K 的加速比分别为 1.067 倍和 1.157 倍，低于稳定强偏置场景，但仍高于 1。其含义是，前半段的适用区间获得了收益，后半段变化及回退承担了一部分成本，整个调用仍保留净收益；这不是后半段也同样提速的证据。

因此，本组结果说明的是“经过筛选的机会在完整接口中有效”，而不只是某条指令执行更快。离线校准用于决定是否值得生成候选，正式结果还包含分发、试测和持续监测，两者不应直接当作同一种加速比。

4.3 无收益负载与回退分析

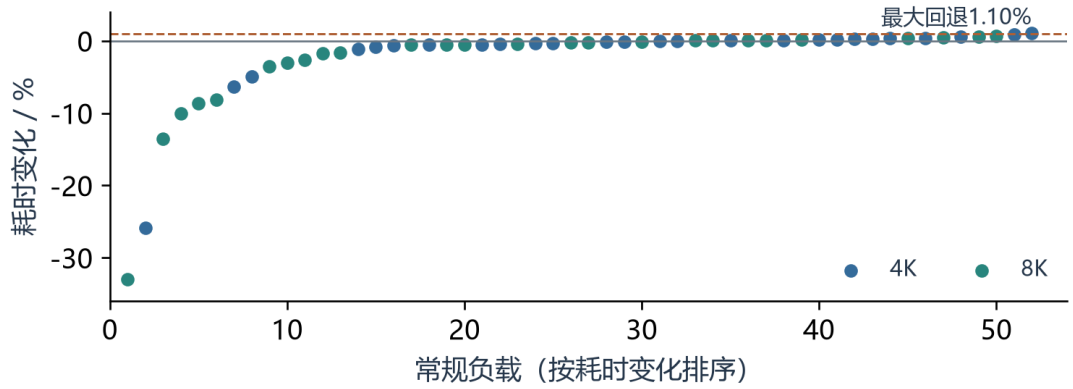


图 4 全部 52 项常规负载的耗时变化，虚线为 1%参考线

为了验证保护机制，我们同时测试均匀输入、短调用、临界批次数和分布切换等场景。52 项常规负载中，51 项的中位回退低于 1%，最大值为 4K threshold513 的 1.10%。这说明额外开销总体较小，但不能将结果表述为任意输入严格零回退。

对于 threshold512/513，又采用新种子和更长计时作独立补测，中位回退分别约 0.17%和 0.21%。补测帮助观察接近 1%的小幅变化，主矩阵中的 1.10%记录仍完整保留。静态不满足条件的程序，则通过可执行节一致性检查验证原路径保持不变。

以均匀查询为例，4K/8K 的加速比分别约 0.999 和 0.996，区间均跨过 1，没有确认稳定收益。两种规模的 62 次优化版测量均未进入持续 B 阶段，试测批数的中位数为 1，读钟次数的中位数为 4。这说明控制器通常在第一对比较后就拒绝了该输入，额外成本来自入口形状检查、这一小段试测和分发。

这里还要区分两种“没有优化”：静态条件不符时，BOLT 不接入新路径；静态条件满足而当前输入不适合时，则需要通过运行时试测才能判断。前者可以通过代码一致性检查，后者必须用完整耗时评价，不能因为最后回到了 A 就认为没有开销。

threshold512/513 位于批次数门槛附近，观察成本能够摊薄的工作量较少，因此更适合暴露准入开销。主矩阵的最大 1.10%按小幅回退报告；独立长计时每次累计四次调用，增加了对细小变化的观察信息，但不会覆盖或替换原结果。

4.4 空表负载性能

表 2 空表调用耗时

空表批数	关闭耗时 / ns	开启耗时 / ns
1	4.51	4.04
512	630.21	4.02
1024	1273.10	4.04

空表开启后约需 4 ns，而关闭版的耗时随批数增加。这一收益来自入口直接消除零工作调用，而不是普通查找变快。因此，我们按绝对耗时单独列出结果，不将其大倍率与非空查找混成平均成绩。

从 1 批增加到 512 批、1024 批时，关闭版仍逐批进入内核，开启版则直接完成整次零结果调用，因而保持在约 4 ns。这个结果同时验证了检查顺序的作用：空表不会先支付普通任务的采样和监测成本，再回头发现没有工作。

4.5 硬件事件分析

为了进一步观察执行成本，我们先用小程序验证 WSL 下的 perf：140 次检查中，9 项必需检查通过。随后对最终二进制的 7 项负载进行 9 轮配对，共 252 次 perf stat，输出全部一致。cycles、instructions、branches 和 branch-misses 四项用户态硬件事件成组采集，运行占比均为 100%。

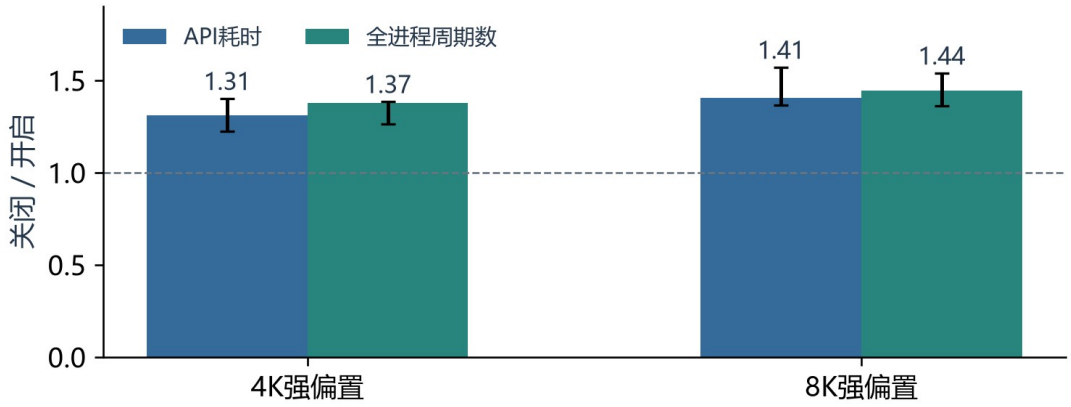


图 5 强偏置负载的 API 耗时比与全进程周期数比

4K/8K 的 API 加速约 1.31/1.41 倍，周期数比约 1.37/1.44，支持强偏置负载在本环境中降低了 CPU 执行成本。硬件事件覆盖整个进程，包括准备和同模式预热；API 计时只覆盖既定调用区间。两种范围分别报告，本次补测也不替代 31 轮正式矩阵。

一个值得关注的现象是：4K 强偏置开启后的指令数和分支失误更多，但总周期数减少。也就是说，性能收益不能简单归因于“执行指令更少”或“预测失误更少”。较好的执行重叠是与前文依赖分析一致的解释，但本次尚未直接测量流水线等待原因。

简单而言，指令数量衡量做了多少操作，周期数还受到这些操作能否重叠执行的影响。转换增加条件分支，即使发生更多预测失误，只要部分后续工作得以更早启动，整体仍可能更快。因此，这里同时观察绝对失误次数、指令数和周期数，而不是仅比较一个失误率。

需要注意的是，图中 API 比与周期数比来自同一轮补测，却覆盖不同范围，不能将二者相减解释为运行时守卫的成本。对守卫开销的主要判断仍来自前面的完整 API 开关对照；硬件事件用于补充解释执行变化。

均匀输入和极短调用的 API 及周期数比接近 1，分布切换场景存在收益；全部 7 项数据保存在补充实验中。采样还确认优化版确实执行了分支副本。针对默认符号显示异常，已按内存映射和 ELF 地址区间重新核对：关闭版 934 个样本中 924 个在原内核，开启版 794 个样本中 460 个在原内核、322 个在分支副本。

采样用于确认程序走过哪些路径，不等同于精确计算各路径耗时占比。WSL 小程序检查支持本环境中所用计数器的一致性，但不能据此推导原生 Linux 或其他处理器必然具有相同加速幅度。

4.6 日志计数应用验证

在受控查找实验之外，我们构建了一个简单的日志时间索引原型：读取 Loghub Linux 2k 公开日志，排序建立时间索引，再计算每个时间阈值之前的事件数，并汇总输出。完整进程时间包含启动、读取与解析、排序、批量计算和输出。

原型使用 2000 条公开日志，每组生成 2097152 条固定种子查询，覆盖范围前查询占 95%、窗口内均匀查询、先旧后新的阶段切换。由于日志没有年份，统一按 2000 年 UTC 解释。训练、校准和测试独立生成；9 轮、162 次进程输出均与 Python bisect 参考一致。

表 3 日志计数应用结果

查询分布	查找加速比	进程加速比	进程 95%区间
95%位于范围前	1.239	1.030	[1.020, 1.042]
窗口内均匀	0.993	1.006	[0.992, 1.017]
先旧后新	1.156	1.024	[1.016, 1.030]

范围前查询的查找部分加速 1.239 倍，完整进程加速 1.030 倍；切换场景的进程加速为 1.024 倍。均匀查询区间跨过 1，未确认收益。这与设计预期一致：机会集中在适合分支执行的输入，而不是所有查询分布。

组件收益与进程收益的差别主要来自时间占比。关闭版中，查找约占完整进程的 16.5%；假设其余时间不变，整体加速约为 1÷(0.835+0.165÷1.239)=1.033 倍，与实测 1.030 倍接近。因此，局部加速明显，而完整应用通常只体现为几个百分点。

对于这三种分布，可以分别理解为：

1.范围前查询：大部分请求早于索引中的最小时间，查找方向稳定，分支版本具有较明确的使用机会。

2.窗口内均匀查询：左右选择随输入变化，缺乏同样的收益条件，控制器拒绝持续 B，结果按未确认收益报告。

3.阶段切换：先旧后新同时包含适用与不适用阶段，用于观察一次调用内的变化，使应用实验对应到持续监测与回退的设计目的。

表中的区间描述固定日志和查询生成规则下的重复计时波动，不包含所有真实用户请求分布的不确定性。范围前场景的进程区间为[1.020,1.042]，支持本次完整流程存在收益；均匀场景区间跨过 1，应按未确认收益理解，不能把中位数略高于 1 直接写成应用提速。

Docker 内独立重建、训练和测量后，范围前查询与切换场景的进程加速分别为 1.020 倍和 1.029 倍，均匀查询仍未确认收益。该原型使用真实日志、合成请求，说明一个简单业务场景中的可用机会，并不等同于生产流量验证。应用源码也可以对范围外查询增加边界判断，本次没有证明 BOLT 方案优于这种算法改写。

数据来源：Loghub，Zhu 等，ISSRE 2023；按随附许可使用。完整数据、独立复现结果和命令见 perf-application.md。

4.7 正确性验证

表 4 功能正确性检查

验证对象	结果
计时输出	正式 14632 次、长计时 992 次全部校验通过。
功能与边界	原始、关闭、开启版 468 次一致；344 个零工作组合通过。
实现与工具	32 项集成、28 项安全、755 项可运行 BOLT 回归通过。
自动流程	Docker 完成 8 项流程检查；校准拒绝产物 156 次功能运行通过。

这些检查分别验证指令改写的功能等价性、零工作判断、异常时的批次推进，以及拒绝路径。性能结果则使用完整 API 或进程时间，将对应范围内的准入、观察和回退成本一并计入。

5 适用范围与局限性

本项目面向 Linux x86-64、有明确批次入口且输入在调用期间只读的程序；只处理满足合法性检查的叶内核和指令模式。其他模式保留原代码。因此，“CMOV 转分支”本身并非新思想，本项目的重点是在 BOLT 中将采样、独立校准、双版本执行和运行时回退结合起来。

当前结果来自一台 WSL 机器，换 CPU 后需要重新校准和计时。成熟应用 Zstd、SQLite 等的历史筛查尚无稳定加速正例，不能用日志原型代替覆盖率验证。下一步应重点增加真实请求分布和处理器测试，记录可用候选及拒绝原因，评价机会出现的频率。

复现方法

准备 Linux x86-64、Bash、Python3 和 Docker，在仓库根目录运行以下命令。入口自动下载并校验镜像；perf 和日志原型按补充说明单独复现。

```
./run.sh                  # 快速流程
./run.sh full             # 完整性能负载
./run.sh functional       # 功能与安全检查
./run.sh evidence         # 复算保存的数据
./run.sh build --source --jobs 2  # 固定源码构建
```

交付位置：代码修改在 stage_1/patches/cmov-runtime.patch；负载在 stage_1/project/cmov_runtime/；原始数据在 stage_1/evidence/；补充实验在 stage_1/experiments/。固定源码提交为 3bd152ddbb6df45e0e07d6c9e9eb1573dcfd3dc6。上游覆盖复核记录截至 2026 年 11 月 12 日。代码与文档整理使用 AI 辅助，数据来自实际运行。